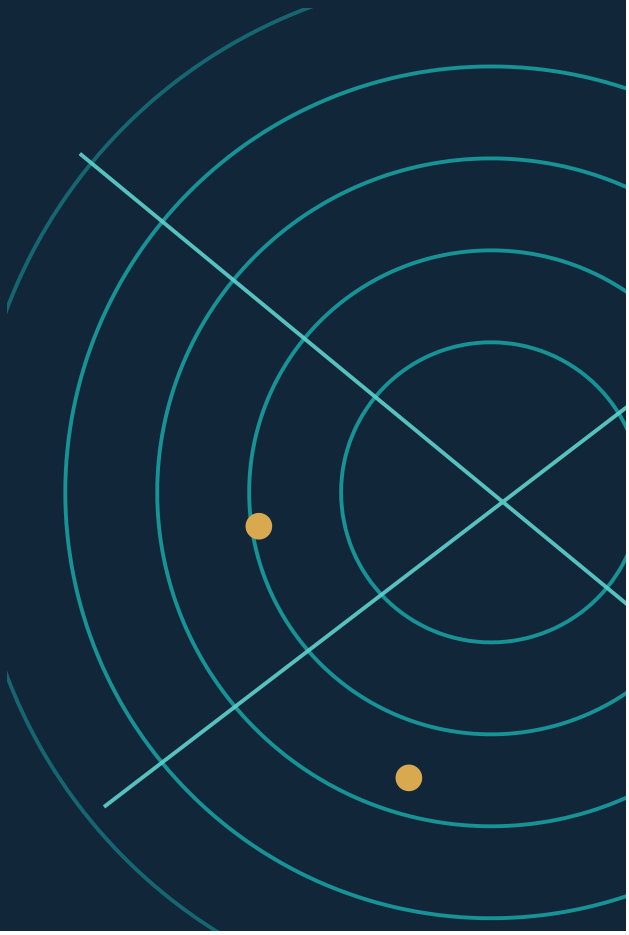
ALGORITHM CHALLENGE 2026 · VERNEWELL GROUP

SphoorthiQ SAR Image Segmentation

Classical and quantum optimization on one verified objective: Markov Random Field, QUBO and QAOA, benchmarked on real MSTAR radar data

THALES USE-CASE PROPOSAL · NOVEMBER 2026



Final proposal submitted for evaluation by the jury and relevant stakeholders of the Algorithm Challenge 2026.

CONTENTS

Contents

1. Executive summary 4
2. Problem and objectives 5
3. Proposed solution 6
4. The running application 8
5. Methodology 12
6. Results to date and expected outcomes 14
7. Supporting documentation and submission details 16
Appendix A. User interface and job execution details 17
Appendix B. How the system works, step by step 20
Bibliographic references 24

List of figures

Figure 1. Solution architecture: one input, two solvers, one score 6
Figure 2. Segmentation result on a real MSTAR chip 8
Figure 3. Metrics and QAOA convergence 9
Figure 4. Pattern detection 9
Figure 5. Synthetic validation 10
Figure 6. Execution record on the quantum platform 11

Where to find what the organizers asked for

Proposed solution · section 3 6
Objectives · section 2 5
Methodology · section 5 12
Expected outcomes · section 6 14
Supporting documentation · section 7 16
Execution details: interface and quantum jobs · Appendix A 17

00 / AT A GLANCE

Document overview

This proposal describes a working application that segments real radar images of vehicles and compares an exact classical solver with a quantum solver on one verified objective. It reports measured results, including where the quantum solver falls short, and states which parts of the THALES brief are only partly met.

DOCUMENT FIELD	DESCRIPTION
Company	SphoorthiQ Private Limited · https://sphoorthiq.com/
Challenge	Algorithm Challenge 2026, organized by Vernewell Group
Use-case	THALES Use-Case: SAR (Synthetic Aperture Radar) Image Segmentation
Submitted to	challenge@vernewellgroup.com (subject: Algorithm Challenge 2026)
Submission deadline	10 November 2026
Team members	Dr. Kalakond Parvath, Dr. Santoshi, Soumyadip
Prepared by	Sangameshwar Swami, SphoorthiQ Private Limited
Document type	Final proposal with measured results
Date	November 2026

Important positioning

All quantum results in this document come from Classiq simulators, not physical quantum hardware, and no quantum advantage is claimed. Real-chip scores use the SARBake masks, which are generated from CAD models: a good reference, not hand-drawn ground truth. On the small tiles where the quantum solver can run, the classical scores are low. The brief's three-class and edge examples, and its 64×64 pre-segmented images, are not reproduced exactly. Each of these points is set out in the sections that follow.

01 / SUMMARY

1. Executive summary

SphoorthiQ is a working application that segments real radar images of vehicles and benchmarks a quantum solver against an exact classical solver on the identical objective. Built for the THALES SAR Image Segmentation use-case, it formulates each image as a Markov Random Field, proves that formulation equivalent to a QUBO, and solves it with both graph cut and QAOA.

On two real chips the quantum solver matches the proven classical optimum at 9 and 16 qubits, and at every noise level tested at 9 qubits. It falls short at 25 qubits. At 9 qubits a random-sampling baseline with the same sample budget finds the optimum too, so that match shows the pipeline is correct, not that QAOA outperforms sampling. At 16 qubits random sampling rarely finds the optimum and at 25 qubits never does, and QAOA's energy is below the random average in all four of those rows. A classical genetic algorithm with the same budget gets closer to the optimum than QAOA at 25 qubits. We report this as measured and do not claim a quantum advantage.

MSTAR has no per-pixel masks, so we scored the classical solvers against the SARBake overlay, whose masks are generated from 3D CAD models of the vehicles. On 120 held-out test chips the two-class target IoU is 0.56 and the three-class mean IoU is 0.66. On the small edge tiles the quantum solver can run on, the classical scores are much lower (target IoU 0.18 on 4×4 tiles), so those scores are better evidence for the pipeline than for the quantum runs.

The proposal's value is a trustworthy benchmark: one verified problem, two solvers, and results that can be reproduced. All quantum results to date ran on Classiq simulators, not physical quantum hardware.

Key facts at a glance

2.13×10^{-14}
largest gap between the MRF and QUBO energies, checked over all 65,536 states of a 4×4 patch

100%
accuracy, IoU and Dice of the exact classical solver on the THALES worked example

53 of 53
automated tests passing, on real MSTAR radar chips with no mocks

0.56
target IoU of the classical solver on 120 held-out real chips, scored against the SARBake masks

51.8%
pattern-detection accuracy on 1,365 real test chips, against 33% by chance

~25 qubits
today's QAOA ceiling on the available simulators: a 5×5 tile for two labels

02 / PROBLEM AND SCOPE

2. Problem and objectives

Synthetic aperture radar (SAR) images must be separated into target and background despite heavy speckle noise [23]. The THALES brief asks for quantum computation to evaluate all pixel-neighbor interactions at once, for a fast and satisfactory segmentation, using a Markov Random Field formulation [20]. State-of-the-art approaches such as clustering and region-based methods [17][18][19] give sub-optimal results, and the brief itself notes that for an Ising prior the exact answer can be computed in polynomial time by graph cut. That sets a high bar, so the core question is how the quantum result compares with the best classical answer.

The brief places the problem in the family of mixed-integer non-linear programs (MINLP) [22], which general-purpose solvers attack mostly by branch-and-bound. Commercial solvers (CPLEX, Gurobi, Xpress, Mosek) and open-source ones (COIN-OR, GEKKO) exist, and the brief notes that their scalability, and the optimality of the answers they return, degrade as the number of variables grows into the thousands. We did not run any of these generic solvers. The brief names a MATLAB genetic algorithm as its baseline. MATLAB is not part of this project, so we implemented a genetic algorithm in Python and ran it on the same energy with the same evaluation budget as QAOA (section 6); it is not MATLAB's ga. For the binary Potts case the structure of the problem also allows the exact answer to be computed in polynomial time by graph cut, so graph cut is the exact classical reference in this work.

Our objectives are:

1. Formulate SAR segmentation as a Markov Random Field and convert it, verifiably, into a form quantum solvers accept.
2. Benchmark QAOA against an exact classical solver on the same objective and the same real data.
3. Measure where the quantum approach holds up and where it stops, by problem size and by noise level.
4. Cover the brief's requirements beyond binary segmentation: images without objects, low signal-to-noise images, edge detection, three or more classes, and pattern detection.
5. Deliver a running application rather than a standalone notebook, so reviewers can inspect each result.

03 / SOLUTION

3. Proposed solution

The solution is a single web application that takes one region of a real SAR image and solves it with two solvers side by side. Because both start from the same problem object and finish on the same energy objective, the comparison is direct rather than approximate.

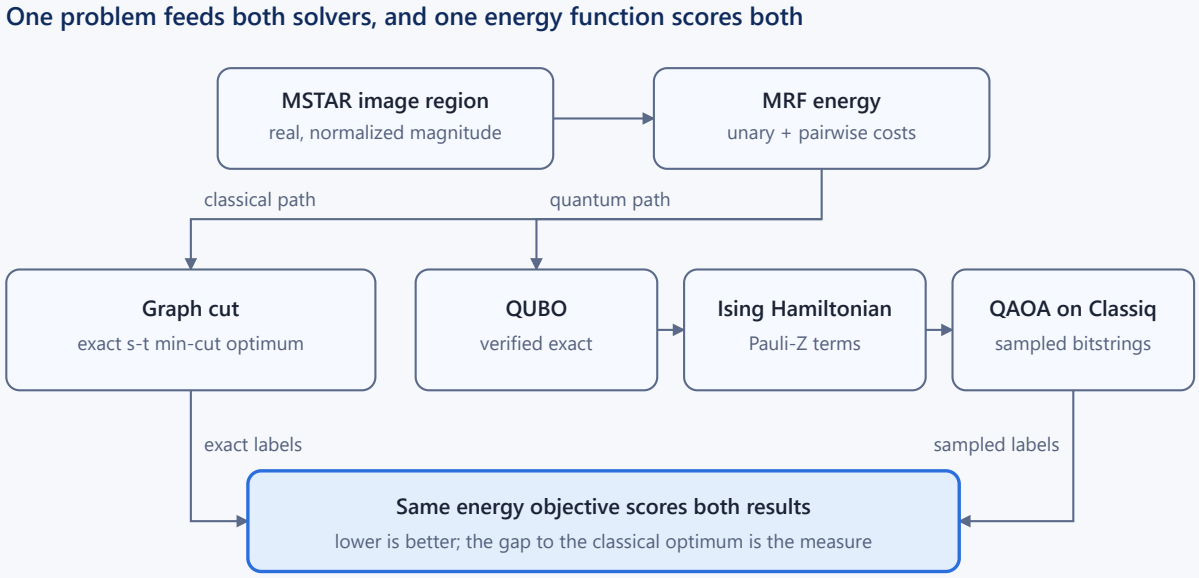


Figure 1. Solution architecture: one input, two solvers, one score.

The app runs as one FastAPI and React process on one port. It streams real per-iteration QAOA progress to the browser, and the classical path works without any quantum account.

Requirement status against the THALES brief

Each requirement in the brief is listed with its status and the caveat that applies. The partly met items are the ones the brief's own data and examples could not support exactly.

BRIEF REQUIREMENT	STATUS	WHAT WE DID, AND THE CAVEAT
Segment SAR images with an MRF, using quantum optimization	**Met**, on simulators	MRF, QUBO, then QAOA on Classiq. Every quantum run used a simulator; none used a physical quantum processor.
Classical baseline (the brief names a MATLAB genetic algorithm)	**Partly met**	Exact graph cut is the classical reference, the method the brief itself names for an Ising prior. We also ran a genetic algorithm that we implemented in Python, because MATLAB is not part of this project. It uses the same energy and QAOA's evaluation budget, and it is not MATLAB's ga. We did not run generic MINLP solvers.
Validation on pre-segmented 64×64 MSTAR images	**Partly met**	The public MSTAR release provides 128×128 chips with no per-pixel masks, so the 64×64 size is not used. Real chips are scored against the SARBake overlay's CAD-derived masks: on 120 held-out chips the classical solver reaches a two-class target IoU of 0.56 and a three-class mean IoU of 0.66. Exact accuracy is also scored on synthetic cases, including the brief's own example.
Binary example (brief, page 3)	**Met** (classical)	Exact graph cut reproduces the printed label matrix. QAOA reaches 92% to 96% accuracy on this 25-qubit example.
Images without an object; low signal-to-noise images	**Met**	Verified target-free and low-contrast region presets on real chips.
Three or more classes (shadow as class 2)	**Partly met**	Background, target and shadow are implemented, with approximate alpha-expansion on the classical side. It does not reproduce the brief's printed three-class example: with a low background reference the solver agrees on 21 of 25 pixels (84%) but labels all four shadow pixels as background, and with a higher reference it agrees on 11 of 25 (44%). The application models a dark shadow, while the example's shadow has mid-level intensity. QAOA is limited to a 2×2 tile. Against the SARBake masks on real chips, the classical three-class result reaches a mean IoU of 0.66, with 0.72 for the shadow.
Edge detection, with the multi-class and single-pixel refinements	**Partly met**	The brief's written rule (any differing neighbor) is implemented, with a single-pixel option and an object-versus-shadow edge map. The brief's printed example marks only the object-side pixel of each boundary; our single-pixel option marks the top or left side, so it differs on that example.
Pattern detection (left undefined)	**Met**, by our interpretation	Target-type recognition across BMP2, BTR70 and T72: 51.79% on 1,365 real test chips, against 33% by chance.

04 / THE APPLICATION

4. The running application

The figures below are screenshots of the application running on real MSTAR data. They were captured in November 2026 from a local run of the application, and the quantum results come from live QAOA runs on Classiq's simulator.

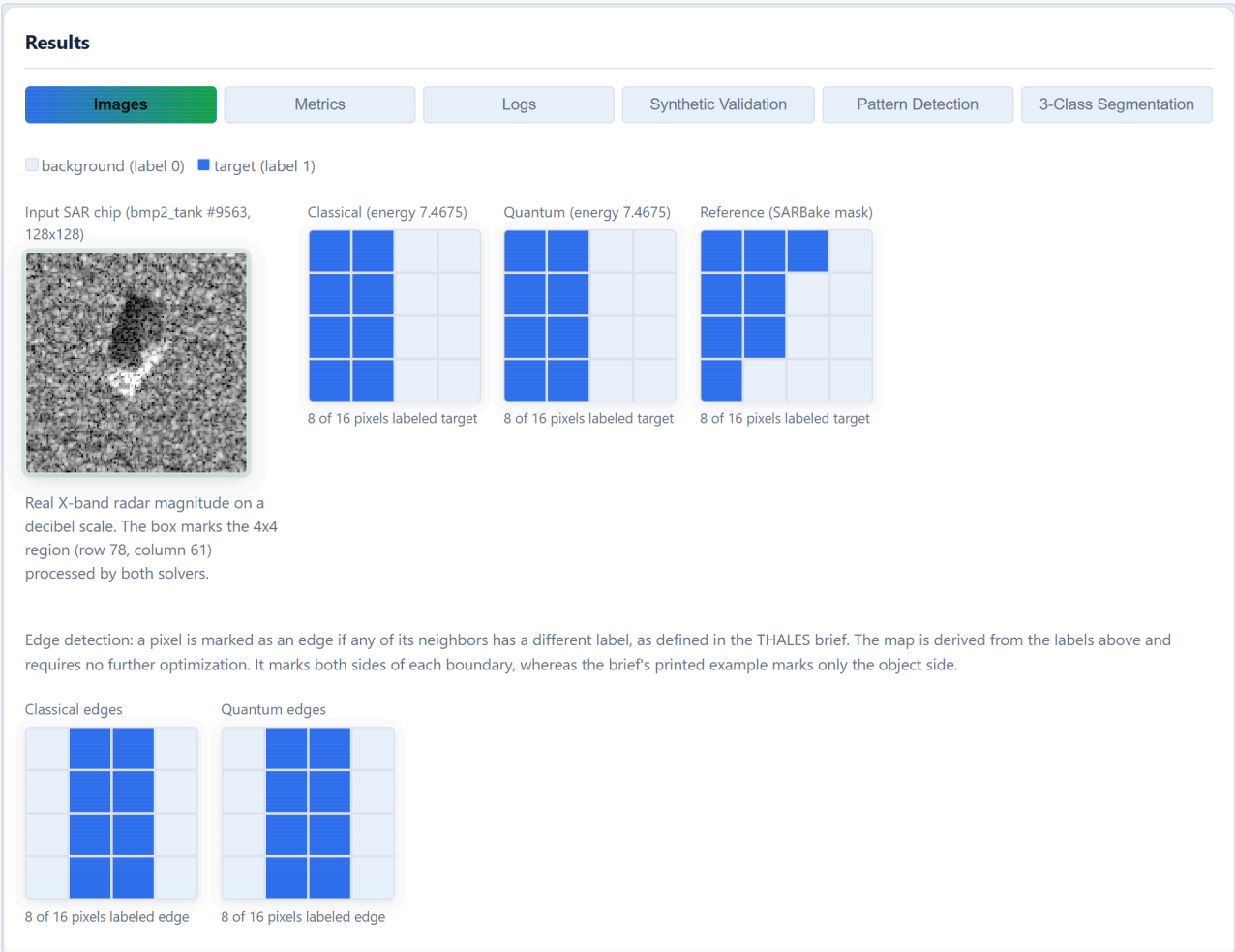


Figure 2. Segmentation result. Real MSTAR chip (BMP2 #9563), 4×4 region (16 qubits) on the vehicle's edge. The panel shows the input chip, the classical segmentation (energy 7.4675), the quantum segmentation (energy 7.4675), the SARBake reference mask, and the derived edge maps. On this run both solvers found the same labeling. QAOA is random, so a repeat run can return a different result, while the classical result is identical every time.

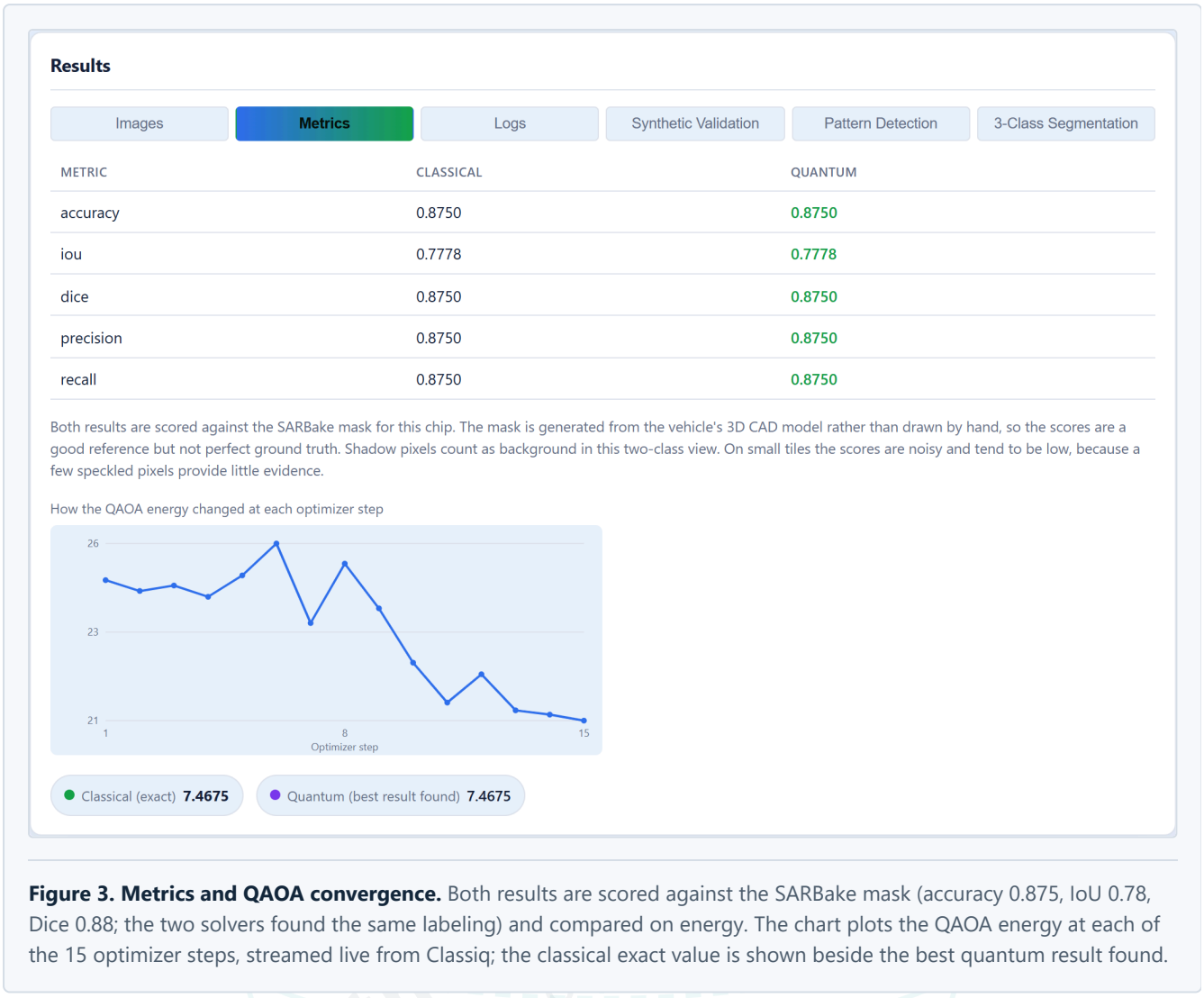


Figure 3. Metrics and QAOA convergence. Both results are scored against the SARBake mask (accuracy 0.875, IoU 0.78, Dice 0.88; the two solvers found the same labeling) and compared on energy. The chart plots the QAOA energy at each of the 15 optimizer steps, streamed live from Classiq; the classical exact value is shown beside the best quantum result found.

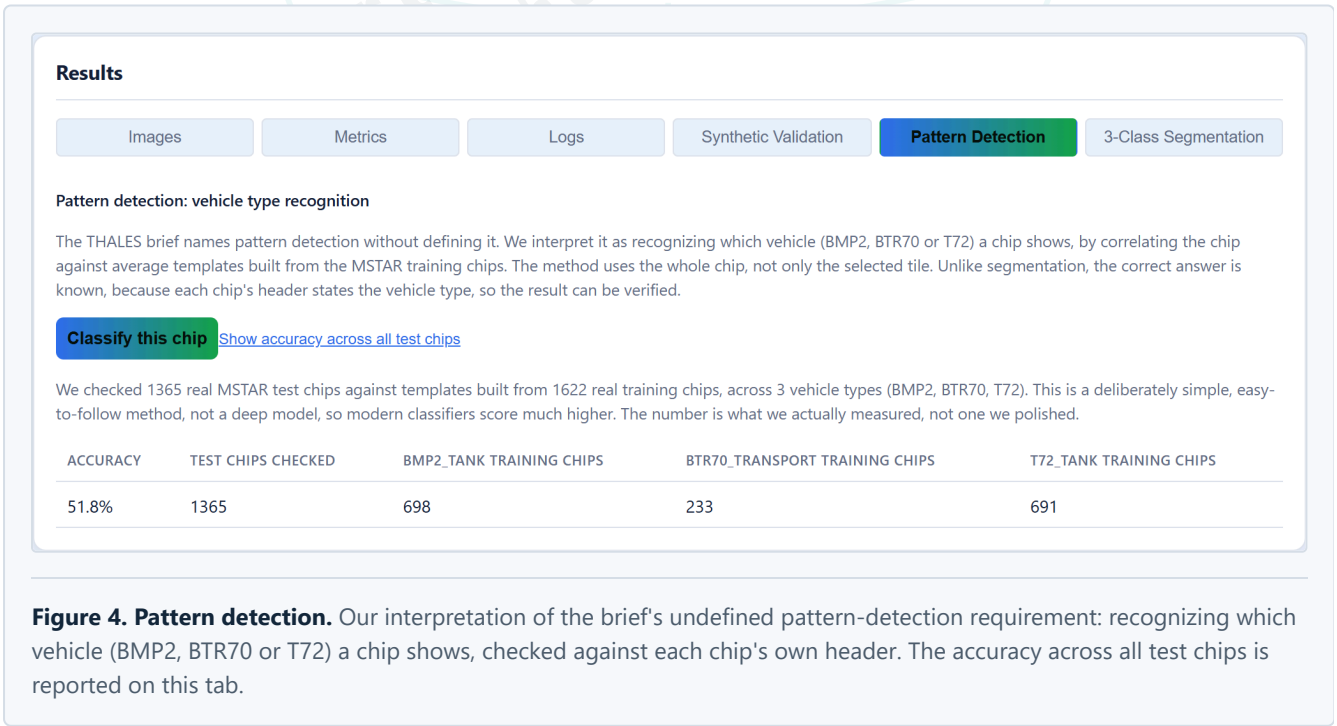


Figure 4. Pattern detection. Our interpretation of the brief's undefined pattern-detection requirement: recognizing which vehicle (BMP2, BTR70 or T72) a chip shows, checked against each chip's own header. The accuracy across all test chips is reported on this tab.



Results

Images | Metrics | Logs | **Synthetic Validation** | Pattern Detection | 3-Class Segmentation

Synthetic validation: exact accuracy scores

These cases are not MSTAR chips. Each has a known correct answer by construction, so accuracy, IoU and Dice are exact. One case is THALES's own worked example, reproduced from the brief; the others are small images generated for this project with a deliberately placed target region. Scores for real chips, against the SARBake masks, are shown in the Metrics tab and on the About page.

☑ Also run the quantum solver on each case (slower; about 60 to 95 seconds each)

Run synthetic validation

These scores are exact because every case has a known correct answer by construction: THALES's own worked example plus a few small cases generated for this project. None of the cases is an MSTAR chip. Real chips are scored against the SARBake masks instead, in the Metrics tab and on the About page.

THALES worked example (5x5) - THALES SAR-segmentation brief, page 3 - quoted directly

METRIC	CLASSICAL	QUANTUM
accuracy	100.0%	96.0%
iou	100.0%	91.7%
dice	100.0%	95.7%
energy	1.4751	5.6723
runtime	0.0055s	138.38s

clean 3x3, corner target - generated: known 2x2 target box

METRIC	CLASSICAL	QUANTUM
accuracy	100.0%	100.0%
iou	100.0%	100.0%
dice	100.0%	100.0%
energy	0.5268	0.5268
runtime	0.0025s	60.59s

clean 4x4, block target - generated: known 2x2 target box

METRIC	CLASSICAL	QUANTUM
accuracy	100.0%	100.0%
iou	100.0%	100.0%
dice	100.0%	100.0%
energy	1.2643	1.2643
runtime	0.0028s	67.57s

noisy 3x3, corner target (sigma=0.15) - generated: known 2x2 target box, synthetic noise sigma=0.15

METRIC	CLASSICAL	QUANTUM
accuracy	100.0%	100.0%
iou	100.0%	100.0%
dice	100.0%	100.0%
energy	0.9571	0.9571
runtime	0.0020s	64.25s

Figure 5. Synthetic validation. Accuracy, IoU and Dice on cases whose right answer is known by construction: the worked example quoted from the THALES brief and three generated cases. These are not MSTAR chips; real chips are scored against the SARBake masks (section 6). This is a fresh run with the quantum solver on: graph cut is exact on all four cases, while QAOA is exact on the three small cases and falls short on the 25-qubit THALES example (96.0% accuracy in this run).

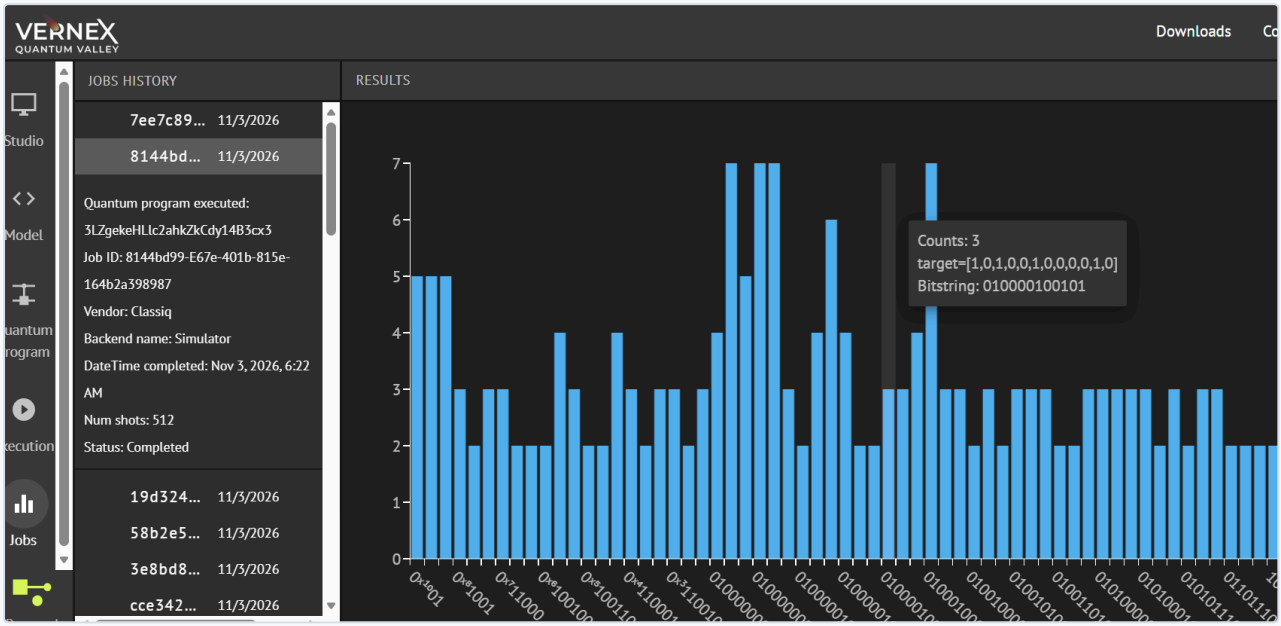


Figure 6. Execution record on the quantum platform. Job history of the cloud platform behind the Classiq SDK, showing a completed run from this project: job ID 8144bd99-e67e-401b-815e-164b2a398987, vendor Classiq, backend Simulator, 512 shots, status Completed, 3 November 2026. The chart is the distribution of measured bitstrings for that job; the tooltip shows one 12-bit outcome (counts: 3). This job is the application's three-class run on BMP2 #9563 (2×2 tile, 12 qubits) recorded at 06:21:36 to 06:22:43 (Appendix A.4), and it confirms that the quantum results were produced by Classiq's simulator, not by physical quantum hardware.

05 / METHOD

5. Methodology

In the MRF view the image is an undirected graph with one vertex per pixel and one edge per pair of neighboring pixels [20]. By Bayes' formula, the posterior probability of a labeling given the image is proportional to the likelihood of the image given the labeling times the prior over labelings. Maximum a posteriori (MAP) inference, which picks the most probable labeling, is equivalent to minimizing an energy.

We minimize one Markov Random Field energy with both solvers and verify each link between them. The energy has a unary term for how well a label fits a pixel's magnitude, and a pairwise term that penalizes disagreeing neighbors [1][3][20]:

$$E(x) = \Sigma_{i \in V} \theta_i(x_i, z_i) + \Sigma_{(i,j) \in E} \Psi_{i,j}(x_i, x_j)$$

The pairwise cost is reduced across real intensity edges, so object boundaries are not smoothed away.

1. **Data.** We use the MSTAR Public Targets chips [15]: X-band SAR images of T72, BMP2 and BTR70 vehicles at 128×128 pixels, plus a SLICY calibration target at 54×54. Pattern detection trains on the 17° depression-angle split and tests on the 15° split, the standard protocol [10][11]. The brief describes pre-segmented 64×64 MSTAR images; the public release provides 128×128 chips with no per-pixel masks. For masks we use the SARBake overlay [21], which labels every T72, BMP2 and BTR70 chip as background, target or shadow from a 3D CAD model of the vehicle at the chip's viewing angle. It is a good reference, not hand-drawn ground truth: it covers the bright vehicle return rather than every pixel a person might outline. SLICY has no masks.
2. **Unary term.** Each pixel's cost comes from its brightness in decibels against its own chip's background, as a robust z-score (median and median absolute deviation of the decibel image). A few constants decide where a pixel starts to count as target and how bright target and shadow are expected to be. We set them on the MSTAR TRAIN chips (17°) and score on the TEST chips (15°), so the reported scores are on chips the constants never saw. There is no trained classifier. An earlier version used the log-scaled, min-max normalized magnitude directly as the target probability; scored against the masks it marked about 13 of roughly 320 target pixels, which is why it was replaced.
3. **Classical solver.** An s-t min-cut graph cut gives the exact optimum for two labels, with no qubit limit [1][2]. It is valid because the pairwise term is a Potts penalty, which is always submodular [4]. The brief names a MATLAB genetic algorithm as its baseline. Graph cut is a stronger reference because it is exact, and we also ran a genetic algorithm implemented in Python (not MATLAB's ga) on the same energy and evaluation budget (section 6).
4. **Quantum solver.** The energy becomes a QUBO, then an Ising Hamiltonian [5], then a QAOA circuit [6] built from Classiq's `qaoa_init` and `qaoa_layer` functions [14]. A COBYLA optimizer [8] tunes the circuit for 15 iterations by default, from a linear-ramp starting point (cf. [7]). Every sampled bitstring is scored on the real QUBO energy, so the quantum result is judged on the same number as the classical one. A final 2,048-shot sample at the tuned parameters picks the best bitstring (Appendix A.2).
5. **Three-class extension.** Background, target and shadow use a one-hot QUBO with a penalty enforcing one label per pixel. Alpha-expansion [3] is the classical baseline, and it is approximate. With 3 qubits per pixel, the 25-qubit ceiling allows a 2×2 tile (12 qubits); a 3×3 tile would need 27.
6. **Pattern detection.** A correlation score compares each chip's target patch with per-class templates averaged from the training split. This is a simple relative of correlation-filter recognition [12], not a reimplementation of it; target-variant recognition on MSTAR is also studied in [13].

Evaluation and design choices

Evaluation uses two kinds of reference. Real chips are scored against the SARBake masks, which are CAD-derived and so a good reference rather than perfect truth, and quantum runs are also compared with the classical optimum on energy. Synthetic cases with known answers, including the worked example in the THALES brief, give exact accuracy, IoU and Dice.

We chose QAOA over quantum annealing, the route used in [9], because Classiq is a gate-model platform. We chose one-hot encoding despite its cost of 3 qubits per pixel, because its validity constraint is simple to verify.

The QUBO step is checked exhaustively: over all 65,536 states of a 4×4 patch, the largest gap between the MRF energy and the QUBO energy is 2.13×10^{-14}, and graph cut matches the brute-force optimum.

06 / RESULTS

6. Results to date and expected outcomes

Graph cut is exact at every size tested. QAOA reaches the classical optimum at 9 and 16 qubits on both chips and falls short at 25 qubits. The table compares the lowest MRF energy found (lower is better) on two real MSTAR test chips, using the default QAOA budget of 15 optimizer iterations. Each tile sits on the vehicle's edge, where about half of it is target according to the SARBake mask, so the solver has a real decision to make. The random-sampling column is a baseline we added: the best of about 9,700 uniformly random labelings, matching QAOA's sample budget, averaged over 30 seeds. The genetic-algorithm column is the mean over 30 seeds of the best energy found by a genetic algorithm implemented in Python (not MATLAB's ga), given the same number of evaluations. The last column scores each result against the mask.

CHIP	QUBITS	CLASSICAL OPTIMUM	QAOA	RANDOM SAMPLING, BEST (MEAN OF 30)	GENETIC ALGORITHM, BEST (MEAN OF 30)	IOU, CLASSICAL / QAOA
T72 #132	9	6.5307	6.5307	6.5307	6.5307	0.40 / 0.40
BMP2 #9563	9	5.3805	5.3805	5.3805	5.3805	0.75 / 0.75
T72 #132	16	11.3321	11.3321	12.1226	11.4294	0.44 / 0.44
BMP2 #9563	16	7.4675	7.4675	8.7559	7.4675	0.78 / 0.78
T72 #132	25	13.8564	14.3462	20.1669	13.9659	0.59 / 0.57
BMP2 #9563	25	12.1758	13.2295	18.4777	12.3871	0.71 / 0.67

The reading is cautious. At 9 qubits there are only 512 labelings, so random sampling finds the optimum on every seed and the match is uninformative. At 16 qubits random sampling found the optimum on only 6 and 3 of 30 seeds, and at 25 qubits on none, and QAOA's energy was below the random average in all four of those rows. The genetic algorithm did better than random sampling at 16 and 25 qubits. It reached the optimum on 27 to 30 of 30 seeds on every tile except the 25-qubit BMP2 tile (18 of 30). At 16 qubits QAOA's single runs also matched the optimum, so the two are level there. At 25 qubits the genetic algorithm's average energy (13.9659 and 12.3871) is below QAOA's (14.3462 and 13.2295). Each QAOA value is a single run, and QAOA is random, so repeat runs can differ. We do not claim a quantum advantage: the classical solver is exact and takes milliseconds. Graph cut also solved 8×8 edge patches (four times QAOA's qubit ceiling) in about 3 ms and 2 ms, with energies of 34.1487 (T72 #132) and 27.4969 (BMP2 #9563) and IoU of 0.71 and 0.78 against the mask.

Real-mask evaluation

We scored the classical solvers on 120 random MSTAR TEST chips (40 each of BMP2, BTR70 and T72, 15° depression), using a 48×48 region around each vehicle. The decibel setup was tuned on the TRAIN chips only. No quantum jobs were run for this table. The masks are CAD-derived, so the scores are a good reference rather than perfect truth.

SCORE ON 120 TEST CHIPS	EARLIER SETUP	DECIBEL SETUP
Two classes: target IoU	0.04	0.56
Two classes: target Dice	0.07	0.70
Two classes: precision / recall	0.99 / 0.04	0.79 / 0.66
Two classes: chips with IoU above 0.5	0 of 120	89 of 120
Three classes: pixel accuracy	0.44	0.82
Three classes: mean IoU	0.16	0.66
Three classes: target / shadow IoU	0.03 / 0.00	0.61 / 0.72

The earlier setup marked about 13 target pixels where the mask has about 320, so nearly all pixels were labelled background. Per vehicle, two-class IoU with the decibel setup is 0.55 for BMP2, 0.50 for BTR70 and 0.62 for T72.

On the small edge tiles the quantum solver can run on, scores are much lower. Each tile is placed where the mask is about half target, the hardest case. Averaged over the same 120 chips, target IoU is 0.12 on 2×2 tiles, 0.18 on 4×4 and 0.34 on 8×8, and three-class pixel accuracy is 0.32 to 0.36, below the 0.50 from always choosing the most common class. The two chips in the scaling table score higher than these averages, so their IoU is not typical.

Other measured results:

- **Noise robustness.** On a 9-qubit edge tile, QAOA matched the classical optimum at all six noise levels from σ = 0 to 0.3, subject to the same caveat about random sampling at that size. Energies cannot be compared between noise levels, because each is a different image. Scores against the mask were 0.40 with no added noise, 0.56 to 0.62 at moderate noise and 0.50 at σ = 0.3, the same for both solvers because they found the same labeling. On a 9-pixel tile one pixel changes IoU substantially, so no trend is inferred.
- **Synthetic validation.** Graph cut reproduces the THALES worked example exactly (accuracy, IoU and Dice all 1.00). QAOA is stochastic and fell short of the classical solver on that 25-qubit example in all four runs we made: accuracy 0.96 (IoU 0.92, Dice 0.96) in two runs and 0.92 (IoU 0.85, Dice 0.92) in the other two. Figure 5 shows the fourth run, at 0.96. It scored 1.00 on the smaller synthetic cases in every run.
- **The brief's other printed examples.** On the three-class example, alpha-expansion agrees with the printed labels on 21 of 25 pixels (84%) when its background reference is 0.2 or lower, but labels all four shadow pixels as background; with a reference of 0.3 or higher it agrees on 11 of 25 (44%). The application models the shadow as dark, while the example's shadow has mid-level intensity. On the edge example, the printed matrix matches marking only the object-side pixel of each boundary; the application's both-sides map differs from it on 5 pixels and its top-or-left single-pixel map on 1.
- **Pattern detection.** 51.79% accuracy on 1,365 real test chips against 33% chance. This is a transparent baseline, well below the roughly 95% reported for CNN-based MSTAR classifiers.
- **Software checks.** All 53 automated tests pass. They run on real MSTAR chips, with no mocks.

Expected outcomes

1. A reproducible benchmark in which two solvers are scored on one verified objective, with repository scripts that regenerate the scaling, robustness and validation tables.
2. An assessment of where quantum optimization stands today: correct but not yet competitive, capped at about 25 qubits on the Classiq simulators.
3. A base for the next stages in our roadmap: tiling a full 128×128 chip into overlapping regions, running on physical quantum hardware, and extending beyond three labels.

07 / SUBMISSION

7. Supporting documentation and submission details

This proposal is submitted to challenge@vernewellgroup.com with the subject line "Algorithm Challenge 2026", by 10 November 2026.

Supporting material in the repository:

- **Algorithm Design Dossier** (`docs/Algorithm_Design_Dossier.md`): problem framing, objective, constraints, assumptions, candidate formulations, scalability, robustness and limitations, section by section.
- **README**: setup, run and deploy steps, the API table, environment variables and known limitations.
- **Scripts**: `scaling_sweep.py`, `robustness_sweep.py` and `synthetic_validation_run.py` regenerate the result tables against a running instance of the app. `sarbake_eval.py` reproduces the real-mask scores and `random_baseline.py` the random-sampling column, and `ga_baseline.py` the genetic-algorithm column; none of the three needs a Classiq session.
- **Tests**: 53 pytest tests on real MSTAR chips, including the exhaustive QUBO equivalence check.
- **Evidence file**: `datasets/reports/11_mrf_qubo_verification.json`.
- **Execution record**: the quantum platform's job history for runs of this application (Figure 6), and the application's own job log (Appendix A).
- **Running application**: a FastAPI backend and React interface in one process, with live QAOA progress.
- **Plain-language explanation**: a step-by-step walkthrough, guidance on reading the results, and a glossary (Appendix B).

Known limits:

- QAOA is capped near 25 qubits, which allows a 5×5 tile for two labels and a 2×2 tile for three.
- All quantum runs used Classiq simulators; no physical quantum processor has been used.
- Real-chip scores use the SARBake masks, which are generated from CAD models, so they are a good guide rather than hand-drawn truth. Chips without a mask (SLICY) are compared on energy only.
- On small edge tiles the scores are much lower (section 6), so the real-mask scores for the quantum runs are weak evidence. The decibel constants were tuned on a small grid. The real-mask scores are for the classical solvers; QAOA was scored only on edge tiles of up to 5×5 pixels.
- Three-class graph cut is approximate, not exact.
- The genetic-algorithm baseline is our own Python implementation, not MATLAB's ga, and its settings were not tuned. The exact classical reference is graph cut.
- The brief's printed three-class and edge examples are not reproduced exactly (section 3).
- The brief describes 64×64 pre-segmented images; we use 128×128 public chips, with masks from SARBake.

A / REFERENCE

Appendix A. User interface and job execution details

This appendix documents how the application is used through its web interface, what happens when a quantum job runs, and the jobs the application has recorded. All figures come from the application's own job history (`datasets/reports/jobs`), read in November 2026. Times are India Standard Time (UTC+5:30).

A.1 Using the web interface

Users sign in, then work in one page with an input panel on the left and result tabs on the right. Every call to the backend carries the signed-in session token. The input panel controls are:

CONTROL	WHAT IT DOES	DEFAULT AND LIMITS
Chip picker	Selects one of 32 catalog chips, grouped by vehicle type and labelled by serial number, azimuth and depression angle: 12 BMP2, 12 T72 and 4 BTR70 (all with SARBake masks) and 4 SLICY (no masks). Choosing a chip moves the region onto the vehicle. `GET /api/mstar/chips`	32 chips; 128×128 pixels (54×54 for SLICY)
Region start row and column	Top-left corner of the tile that both solvers process. The chip view shows the tile as a box.	60 and 60, then set by the chip choice
Region presets	*Target* centres the tile on the vehicle, *No object* on a verified target-free area, and *Low SNR* on a verified low-contrast area, the three cases the brief asks for.	Use the current tile size
Tile size	Pixels per side. The label shows how many qubits the tile needs if it is run on the quantum solver.	4 (16 qubits); 1 to 12; quantum limited to 5 (25 qubits)
Also run the quantum solver	Adds QAOA on Classiq's simulator next to the classical result. A tile that is too large for QAOA shows a message and disables the run.	On
Advanced options	Smoothness (strength of the neighbor agreement term), edge sensitivity (contrast sigma), added noise for robustness tests, QAOA layers, maximum optimizer steps, and shots per step.	1.0; 0.15; 0; 2; 15; 512

The result tabs are:

VIEW	WHAT THE USER DOES AND SEES	BACKEND CALL
Images	Runs the pipeline with the controls above. Sees the chip with the tile boxed, the classical and quantum labelings with their energies, the SARBake reference mask, and the edge maps (both sides of each boundary, and one pixel per boundary). During a quantum run a live panel shows the classical result and each QAOA step as it completes.	`POST /api/mstar` or `/api/mstar/stream` (live QAOA progress)
Metrics	Reads the classical and quantum energies, their scores against the reference mask (accuracy, IoU, Dice, precision and recall), and the QAOA convergence chart.	Same response as the run

VIEW	WHAT THE USER DOES AND SEES	BACKEND CALL
Logs	Reads the run log for the current result: classical and quantum durations, qubit count, circuit width and depth, layers, shots, backend, and the optimized circuit parameters.	Same response as the run
Synthetic Validation	Runs the THALES worked example and three generated cases, optionally with the quantum solver (about 60 to 95 seconds per case). Reads exact accuracy, IoU, Dice, energy and runtime for each case.	`POST /api/mstar/synthetic-validation`
Pattern Detection	Classifies the selected chip and shows its true type, the predicted type and the match score for each class, or runs the accuracy check across all 1,365 test chips (about 30 seconds).	`POST /api/mstar/pattern`, `GET /api/mstar/pattern/evaluation`
3-Class Segmentation	Segments the tile into background, target and shadow, optionally with the quantum solver on tiles of 2×2 or smaller (the tab shows the qubits the tile needs). Shows the SARBake reference mask, the scores against it, and edge maps by type: object, shadow, and object-shadow.	`POST /api/mstar/shadow`
About page	Reads the workflow, the steps of a quantum run, verification evidence, scaling, robustness, validation and beyond-binary results.	None (static page)

A.2 What happens when a quantum job runs

1. The browser sends the chip, region and QAOA settings. The defaults are 2 layers, up to 15 optimizer iterations and 512 shots per iteration.
2. The server reads the real MSTAR region, builds the MRF energy and solves it exactly by graph cut, which takes milliseconds.
3. It converts the energy to a QUBO and an Ising Hamiltonian; Classiq's cloud synthesizes the QAOA circuit.
4. Each optimizer iteration submits one 512-shot job to Classiq's simulator. Every sampled bitstring is scored on the QUBO energy, and the progress streams back to the interface.
5. A final 2,048-shot sample at the optimized parameters picks the best bitstring seen.
6. The server returns the quantum result beside the classical one, scored against the SARBake mask when one exists, and records the run in the job history.

Each optimizer iteration is therefore its own Classiq job, which is why a run takes about a minute. The platform record in Figure 6 is one such 512-shot job.

A.3 Job history recorded by the application

The history includes the project's own automated tests, sweeps and verification runs as well as manual use of the interface, so it counts repeat runs of the same chip. A.3 and A.4 cover 16 September to 3 November 2026, before the intensity update in section 5, so their energies are not comparable with section 6.

MEASURE	BINARY SEGMENTATION	THREE-CLASS SEGMENTATION
Jobs recorded, 16 September to 3 November 2026	248	87
Of which with QAOA on Classiq's simulator	102	23
Recorded as successful	248	87
QAOA duration, median (range)	66.6 s (26.4 to 319.2 s)	66.8 s (63.5 to 78.8 s)

For the 102 binary QAOA jobs, the table shows how often QAOA reached the exact classical optimum (energy within 0.001) by tile size. They cover 28 different chips.

TILE	QUBITS	QAOA JOBS	MATCHED THE CLASSICAL OPTIMUM
1×1	1	6	6
2×2	4	2	2
3×3	9	11	11
4×4	16	66	17
5×5	25	17	0

A.4 Jobs run on 3 November 2026

START (IST)	VIEW	CHIP	TILE	QUBITS	CLASSICAL ENERGY	QAOA ENERGY	DURATION (S)
06:13:20	Binary	bmp2_9563_test_1	4×4	16	5.6416	6.0188	81.9
06:21:36	Three-class	bmp2_9563_test_1	2×2	12	0.4259	0.4259	67.3
06:24:28	Binary	slicy_15deg_1	5×5	25	6.178	12.7708	130.9
06:26:24	Binary	bmp2_9563_test_1	4×4	16	5.6416	7.0123	67.0
06:29:03	Binary	bmp2_9563_test_1	4×4	16	5.6416	6.4645	64.4
06:34:22	Three-class	slicy_15deg_1	2×2	12	0.4905	0.4905	63.8

The Classiq platform record in Figure 6 (completed 6:22 AM, 512 shots, 12-bit outcomes, job ID 8144bd99-e67e-401b-815e-164b2a398987) corresponds to the 06:21:36 three-class run on BMP2 #9563, by completion time and qubit count. The three runs of the same 4×4 tile returned QAOA energies of 6.0188, 7.0123 and 6.4645 against one classical optimum of 5.6416, which shows run-to-run variation in QAOA.

After the intensity update (4 November 2026). The section 6 sweeps ran 12 QAOA jobs on Classiq's simulator (six tile sizes, six noise levels), plus the run in Figures 2 and 3. All succeeded, in 60.0 to 127.5 s.

B / EXPLANATION

Appendix B. How the system works, step by step

This appendix explains the system in plain terms, for a reader who is not a specialist in radar, optimization or quantum computing. It follows one real example through every stage, explains how to read the results, and defines the terms used in this document.

B.1 The problem

A radar image of a vehicle is noisy: speckle makes individual pixels unreliable. The task is to label every pixel as background (class 0) or vehicle (class 1), and optionally shadow (class 2). A good labeling agrees with the evidence in each pixel (is it bright like a vehicle?) while staying consistent with its neighbors (a pixel beside vehicle pixels is probably vehicle too). The system writes this trade-off as a single number, the **energy**, and looks for the labeling with the lowest energy. Both solvers minimize the same energy, so their results can be compared directly.

B.2 One example, step by step

The example is the run in Figures 2 and 3: chip BMP2 #9563, a 4×4 tile (16 pixels) with its top-left corner at row 78, column 61.

STEP	WHAT HAPPENS	WHERE IN THE CODE
1	**Choose a tile.** The 16 pixels are cut from the 128×128 chip.	`utils/mrf/mstar_scenes.py`
2	**Measure brightness against the background.** Each pixel's magnitude is converted to decibels. The chip's median is subtracted and the result is divided by a robust spread (1.4826 times the median absolute deviation), giving a z-score. The intensity used for edges is (z + 4) / 12, limited to 0 to 1.	`calibrate_magnitude` in `mstar_scenes.py`
3	**Cost of each label for each pixel (unary term).** The probability that a pixel is target is $1 / (1 + e^{-(z - 0.8)})$. Labeling the pixel target costs −log of that probability, and labeling it background costs −log of the remainder. A bright pixel therefore makes "target" cheap and "background" expensive.	`build_patch_energy` in `utils/mrf/energy.py`
4	**Cost of disagreement between neighbors (pairwise term).** For every pair of horizontally or vertically adjacent pixels, a penalty is paid if their labels differ. The penalty is the smoothness setting (1.0) times $e^{-d^2 / (2 \times 0.15^2)}$, where d is the difference in intensity, so it shrinks across a strong brightness edge and object boundaries are not smoothed away.	same function
5	**Energy of a labeling** = the unary costs of the chosen labels + the penalties of all disagreeing neighbor pairs. The goal is the labeling with the lowest energy.	`energy` in `energy.py`
6	**Classical solution.** Graph cut (a minimum cut on a graph built from the costs) finds the lowest possible energy exactly, in milliseconds. Here the energy is 7.4675.	`solve_graph_cut`
7	**Quantum formulation.** The same energy is rewritten as a quadratic function of 16 binary variables (a QUBO) and then as an Ising Hamiltonian, with one qubit per pixel. The rewrite was checked against the original energy on all 65,536 labelings of a 4×4 patch.	`build_qubo`, `utils/mrf/verification.py`

STEP	WHAT HAPPENS	WHERE IN THE CODE
8	**QAOA on Classiq.** A circuit with 2 layers, built from Classiq's `qaoa_init` and `qaoa_layer`, runs on 16 qubits. A classical optimizer (COBYLA) adjusts the 4 circuit angles for up to 15 steps. Each step runs 512 shots on Classiq's simulator, and every sampled bitstring is scored on the true energy. A final 2,048-shot sample at the best angles is taken, and the lowest-energy bitstring seen becomes the quantum labeling. Here its energy is also 7.4675.	`utils/qml/classiq_qaoa.py`
9	**Scoring against the reference mask.** Each labeling is compared with the SARBake mask for the same tile: accuracy 0.875, IoU 0.78 and Dice 0.88. The two solvers scored the same because they found the same labeling.	`utils/mrf/sarbake.py`
10	**Edge maps.** A pixel is an edge if any neighbor has a different label. The maps are derived from the labels and need no further optimization.	front end, `MrfPage.tsx`

B.3 Three classes

With three labels a pixel can no longer be described by one binary variable. Each pixel uses three qubits, one per label, and a penalty term forces exactly one of the three to be on (a one-hot encoding). Graph cut works only for two labels, so the classical baseline for three is alpha-expansion, a sequence of two-label graph cuts that is approximate. The unary cost is the squared distance between the pixel's z-score and the expected z-score of each class (background 0, target 1.5, shadow −1.0), divided by a scale of 3. The tile limit for the quantum solver falls from 5×5 to 2×2 (12 qubits) because of the extra qubits per pixel.

B.4 How to read the results

- **Energy** is the optimization objective: lower is better. Energies can be compared between the two solvers on the same tile, but not between different tiles or noise levels, because each is a different problem.
- **Accuracy, IoU and Dice** measure agreement with the reference mask. Accuracy is the share of pixels labeled correctly. IoU (intersection over union) is the overlap of the predicted and true target regions divided by their combined area. Dice is twice the overlap divided by the sum of the two areas. IoU and Dice ignore the large background, so they are stricter than accuracy.
- **Lowest energy is not the same as best match to the mask.** The energy uses brightness only, and the mask comes from a CAD model. A labeling can have a higher energy and still match the mask as well or better, so the two solvers are compared on energy, with the mask scores as a second view.
- **QAOA is random.** Repeating a run on the same tile can give a different result, so each QAOA value in this document is a single run unless stated otherwise.
- **All quantum runs use a simulator.** No physical quantum hardware has been used. The simulators handle about 25 qubits, which limits the tile size, and a whole 128×128 chip would have to be split into many small tiles.
- **Small tiles score lower.** A tile of a few pixels gives little evidence for telling target from background, which is why the real-mask scores on small edge tiles are much lower than on larger regions (section 6).

B.5 Glossary

TERM	MEANING
SAR	Synthetic aperture radar: an imaging radar that produces pictures of the ground. Its images contain speckle, a grainy noise that makes single pixels unreliable.
MSTAR	A public set of SAR images of military vehicles (T72, BMP2, BTR70 and a SLICY calibration object), released by the U.S. Air Force Research Laboratory. Each image is a "chip" of 128×128 pixels (54×54 for SLICY).
Tile (region of interest)	The small square of a chip that the solvers process. Its size sets the number of qubits needed.
Markov Random Field (MRF)	A model in which each pixel has a label and the labels of neighboring pixels influence each other. It turns segmentation into minimizing an energy.
Unary and pairwise terms	The two parts of the energy: the cost of a label at one pixel given its brightness, and the cost of two neighboring pixels disagreeing.
Potts penalty	A pairwise cost paid when two neighbors have different labels and nothing otherwise. It is what makes exact graph cut possible for two labels.
Graph cut	A classical method that finds the exact lowest-energy labeling for two labels by computing a minimum cut on a graph. It has no qubit limit.
Alpha-expansion	A classical method for three or more labels that repeats two-label graph cuts. It is approximate.
QUBO	Quadratic unconstrained binary optimization: minimizing a quadratic function of binary variables. It is the form quantum optimizers accept.
Ising Hamiltonian	The physics form of a QUBO, with variables of +1 or −1. It is the form a QAOA circuit encodes.
One-hot encoding	Using one binary variable per label per pixel, with a penalty so that exactly one is on. It costs three qubits per pixel for three labels.
QAOA	Quantum approximate optimization algorithm: a circuit of alternating layers whose angles are tuned by a classical optimizer to make low-energy bitstrings more likely. It searches for a good answer and does not guarantee the best.
Classiq, Qmod	The quantum software platform and its modeling language used to build, synthesize and run the QAOA circuit.
Qubit, shot	A qubit is the quantum counterpart of a bit. A shot is one run of the circuit that yields one bitstring; many shots are taken per step.
COBYLA	The classical optimizer that tunes the circuit angles without needing gradients.
Genetic algorithm	A search method that evolves a population of candidate labelings through selection, crossover and mutation. Here it is the classical baseline the brief names, implemented in Python rather than MATLAB.
Simulator	Software that mimics a quantum computer. All quantum results in this document come from Classiq simulators.
Decibel z-score	A pixel's brightness in decibels, minus the chip's median, divided by a robust spread. It states how far a pixel stands above or below the background.
SARBake mask	A reference label (background, target or shadow) for every pixel of an MSTAR chip, generated from a 3D CAD model of the vehicle. It is a good reference but not hand-drawn ground truth.
IoU, Dice, accuracy	Scores of agreement with a reference mask, defined in B.4.

TERM	MEANING
TRAIN and TEST split	MSTAR's two groups of chips (17° and 15° depression angle). Constants were set on TRAIN and results are scored on TEST, so the scores are on chips the constants never saw.
Synthetic case	A small test image built so that the correct labeling is known, used for exact accuracy scores. None is an MSTAR chip.

REFERENCES

Bibliographic references

Numbers match the citations in the text. All seven references listed in the THALES brief [16] are included here, as 9, 17, 18, 19, 20, 22 and 23.

1. D. M. Greig, B. T. Porteous and A. H. Seheult (1989). Exact maximum a posteriori estimation for binary images. *Journal of the Royal Statistical Society: Series B* 51(2), 271–279. doi:10.1111/j.2517-6161.1989.tb01764.x
2. Y. Boykov and M.-P. Jolly (2001). Interactive graph cuts for optimal boundary and region segmentation of objects in N-D images. *Proc. IEEE ICCV*, vol. I, 105–112. doi:10.1109/ICCV.2001.937505
3. Y. Boykov, O. Veksler and R. Zabih (2001). Fast approximate energy minimization via graph cuts. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 23(11), 1222–1239. doi:10.1109/34.969114
4. V. Kolmogorov and R. Zabih (2004). What energy functions can be minimized via graph cuts? *IEEE Transactions on Pattern Analysis and Machine Intelligence* 26(2), 147–159. doi:10.1109/TPAMI.2004.1262177
5. A. Lucas (2014). Ising formulations of many NP problems. *Frontiers in Physics* 2, 5. doi:10.3389/fphy.2014.00005
6. E. Farhi, J. Goldstone and S. Gutmann (2014). A quantum approximate optimization algorithm. arXiv:1411.4028. arxiv.org/abs/1411.4028
7. S. H. Sack and M. Serbyn (2021). Quantum annealing initialization of the quantum approximate optimization algorithm. *Quantum* 5, 491. doi:10.22331/q-2021-07-01-491
8. M. J. D. Powell (1994). A direct search optimization method that models the objective and constraint functions by linear interpolation. In S. Gomez and J.-P. Hennart (eds), *Advances in Optimization and Numerical Analysis*, Kluwer Academic, 51–67. (The COBYLA method.)
9. S. Otgonbaatar and M. Datcu (2020). Quantum annealing approach: feature extraction and segmentation of synthetic aperture radar image. *Proc. IEEE IGARSS 2020*, 3692–3695. doi:10.1109/IGARSS39084.2020.9323504
10. E. R. Keydel, S. W. Lee and J. T. Moore (1996). MSTAR extended operating conditions: a tutorial. *Proc. SPIE* 2757, 228–242. doi:10.1117/12.242059
11. T. D. Ross, S. W. Worrell, V. J. Velten, J. C. Mossing and M. L. Bryant (1998). Standard SAR ATR evaluation experiments using the MSTAR public release data set. *Proc. SPIE* 3370, 566–573. doi:10.1117/12.321859
12. A. Mahalanobis, B. V. K. Vijaya Kumar, S. Song, S. R. F. Sims and J. F. Epperson (1994). Unconstrained correlation filters. *Applied Optics* 33(17), 3751–3759. doi:10.1364/AO.33.003751
13. B. Bhanu and G. Jones III (2000). Recognizing target variants and articulations in synthetic aperture radar images. *Optical Engineering* 39(3), 712–723. doi:10.1117/1.602418
14. Classiq Technologies. Classiq SDK and Qmod open library (`qaoa_init`, `qaoa_layer`). classiq.io
15. U.S. Air Force Research Laboratory. MSTAR Public Targets Chips (T72, BMP2, BTR70, SLICY), Sensor Data Management System. sdms.afrl.af.mil
16. THALES. *THALES Use-Case: SAR (Synthetic Aperture Radar) Image Segmentation*. Use-case brief supplied to participants (4 pages).
17. S. Huang, W. Huang and T. Zhang (2016). A new SAR image segmentation algorithm for the detection of target and shadow regions. *Scientific Reports* 6, 38596. doi:10.1038/srep38596
18. R. Shang, J. Lin, L. Jiao and Y. Li (2020). SAR image segmentation using region smoothing and label correction. *Remote Sensing* 12(5), 803. doi:10.3390/rs12050803
19. F. Galland, J.-M. Nicolas, H. Sportouche, M. Roche, F. Tupin et al. (2009). Unsupervised synthetic aperture radar image segmentation using Fisher distributions. *IEEE Transactions on Geoscience and Remote Sensing* 47(8), 2966–2972. HAL: hal-00437471.
20. H. Kunsch, S. Geman and A. Kehagias (1995). Hidden Markov random fields. *Annals of Applied Probability* 5(3), 577–602. doi:10.1214/aoap/1177004696
21. D. Malmgren-Hansen and M. Nobel-Jørgensen (2015). Convolutional neural networks for SAR image segmentation. *Proc. IEEE ISSPIT 2015*. Masks: *SARBake Overlays for the MSTAR Dataset*, Mendeley Data, v3 (2017), CC BY 4.0. doi:10.17632/jxhsg8tj7g.3
22. P. Belotti et al. (2012). *Mixed-Integer Nonlinear Optimization*. Preprint ANL/MCS-P3060-1112, Mathematics and Computer Science Division, Argonne National Laboratory, Argonne, Illinois, USA.
23. Synthetic Aperture Radar. radartutorial.eu/20.airborne/ab07.en.html